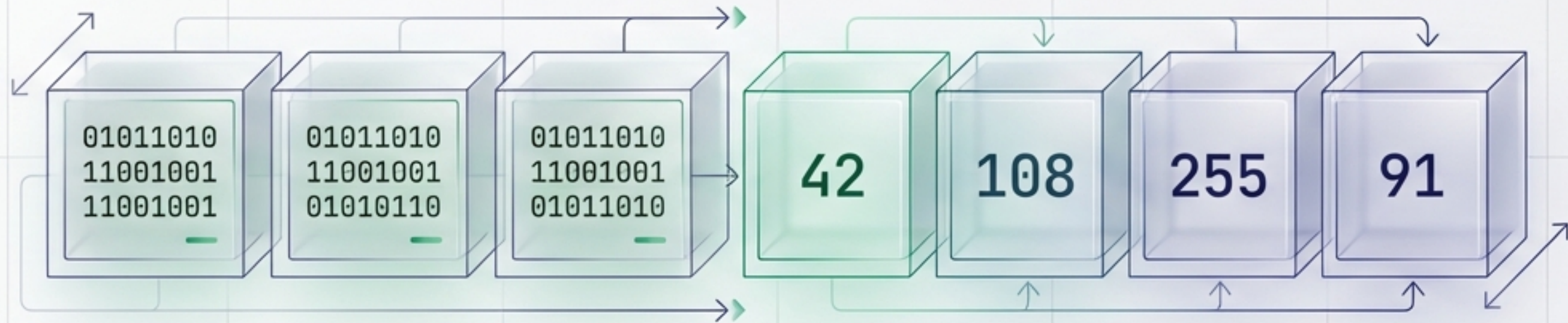




Mastering Arrays in Python

The Foundation of Data Structures & Algorithms

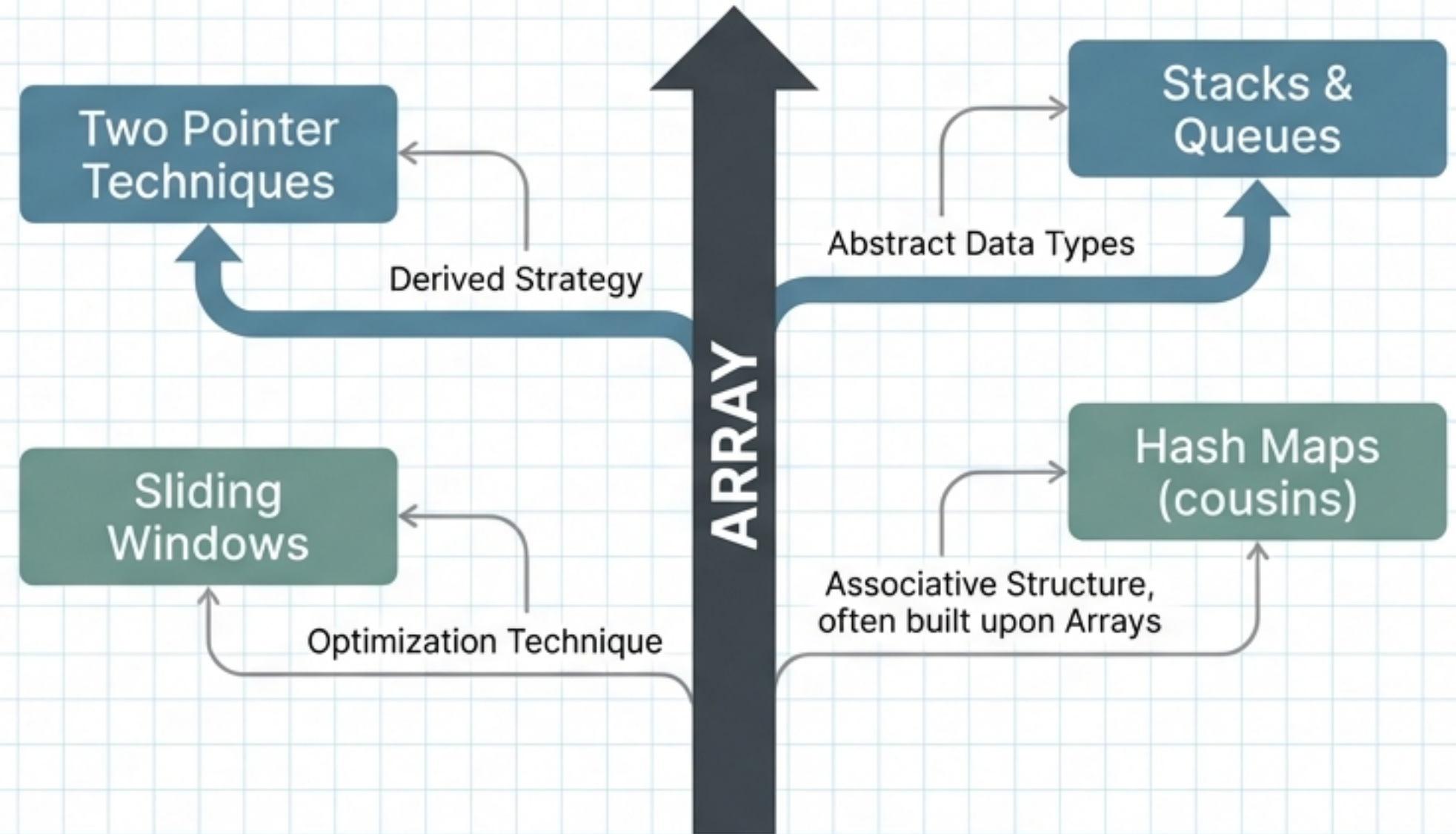


From raw memory management to multi-dimensional Numpy matrices.

The Root of Algorithmic Logic

90%

of Data Structures & Algorithms challenges rely on **Arrays**

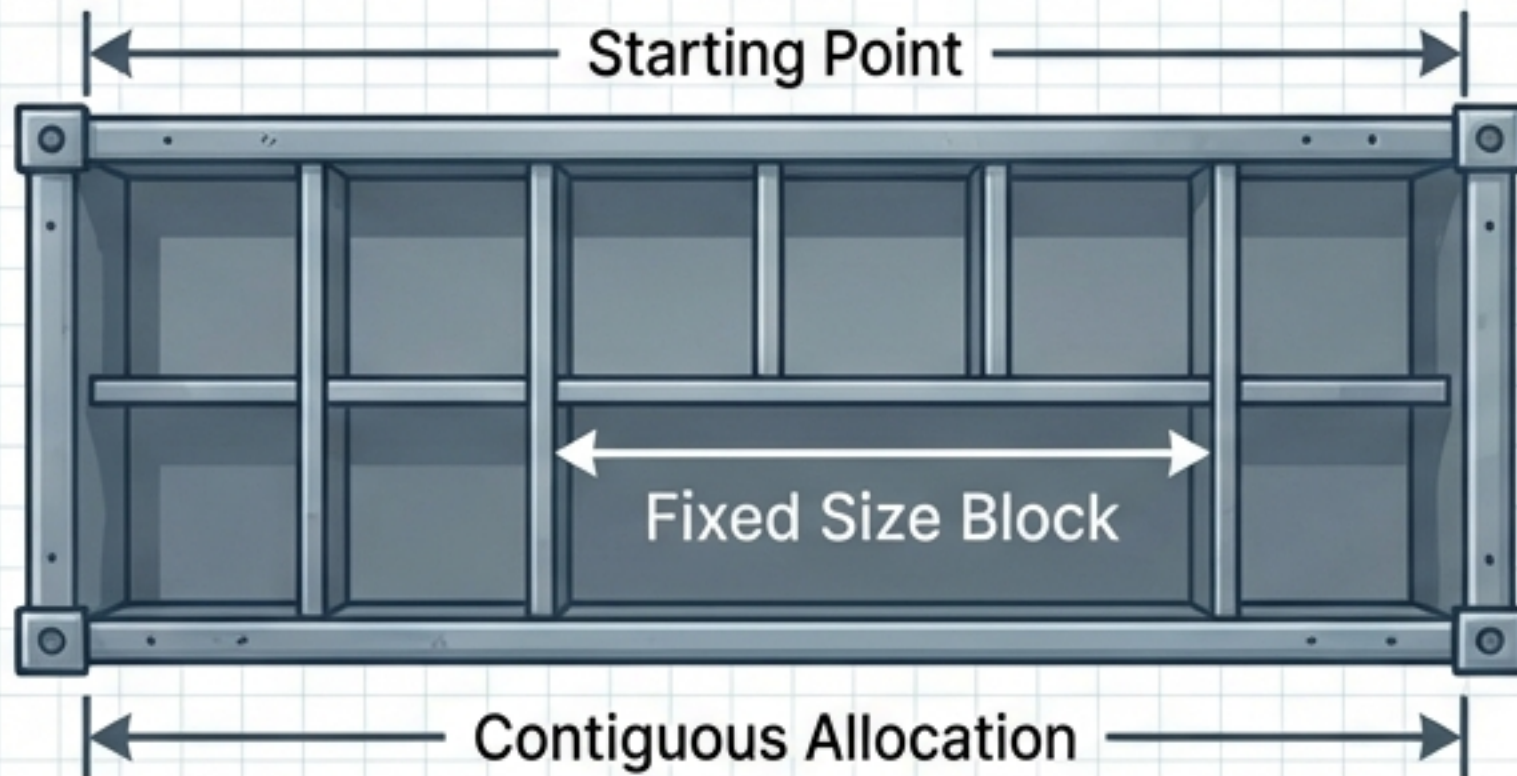


Real-world implementation is effectively **100%**. Whether managing dynamic sizes or fixed memory blocks, the array is the starting line for solving complex logic problems.

Visualizing the foundational hierarchy of computational structures.

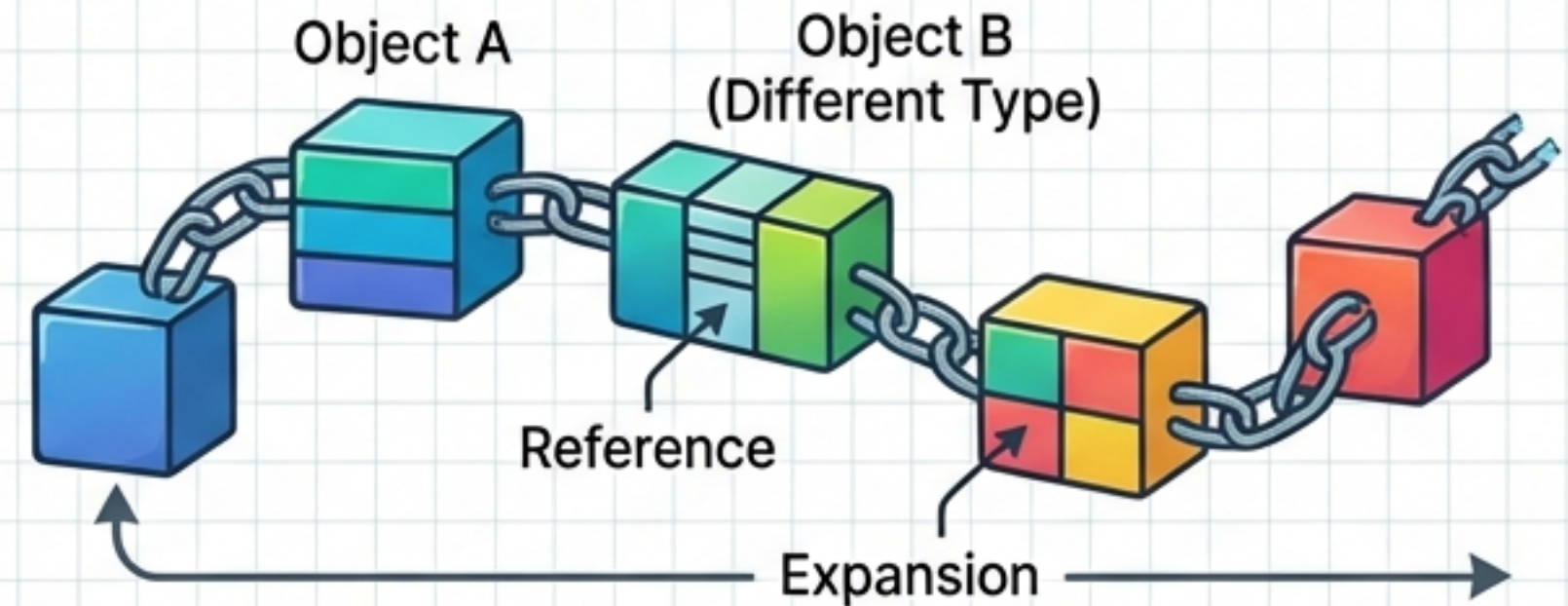
Concept vs. Reality: The Python Difference

Traditional (C / Java)



- Contiguous memory allocation
- Strict Homogeneity (Same Type)
- Fixed Size

Python Implementation

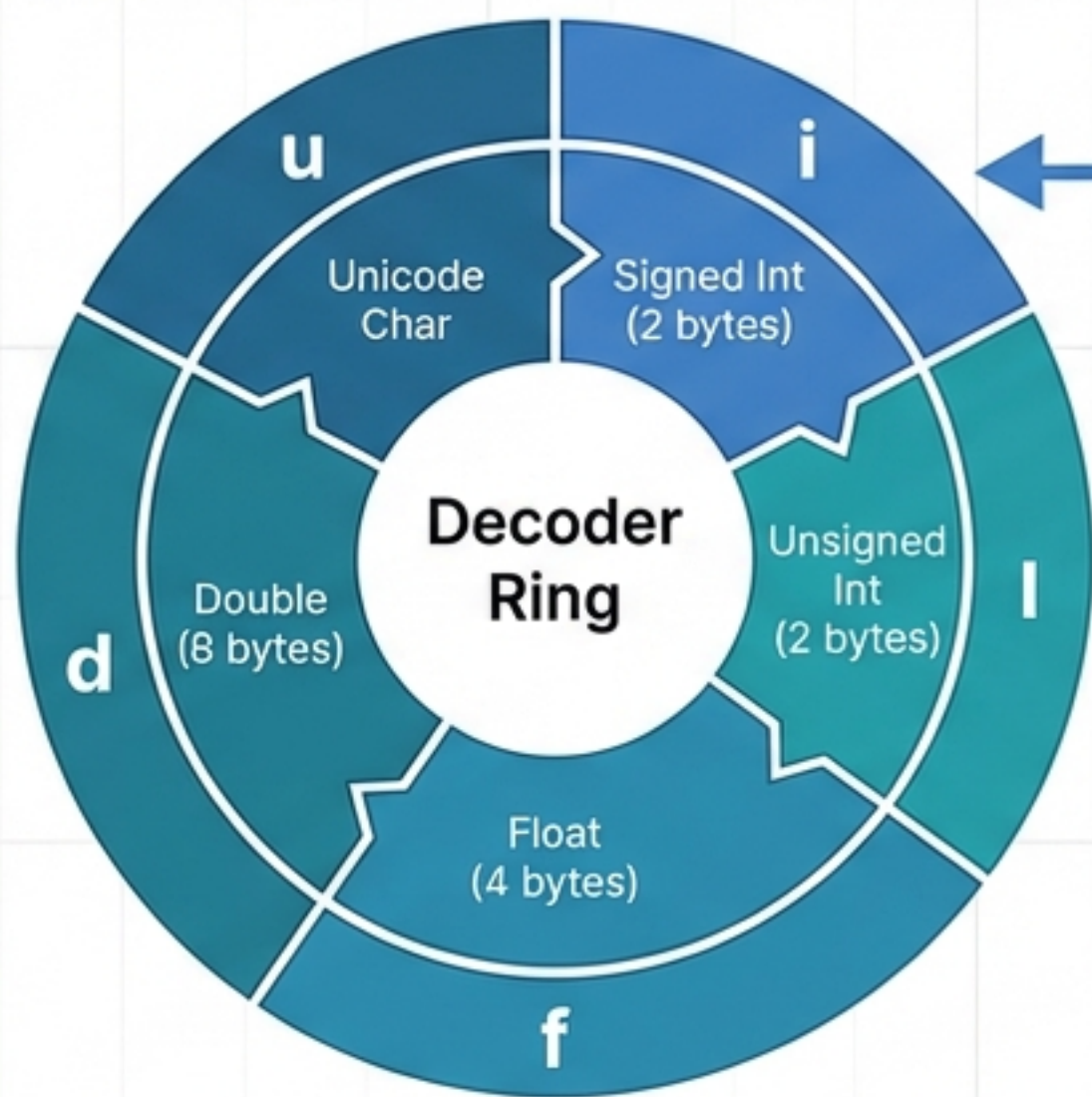


- Dynamic Sizing (Auto-memory)
- Heterogeneous Potential (Lists/Numpy)
- Reference-based

Insight: In Python, you don't manually manage memory blocks, but understanding the underlying contiguous allocation is vital for optimization.

The Native Solution: The 'array' Module

Strict types for memory efficiency.



Decoder Ring

```
import array as arr

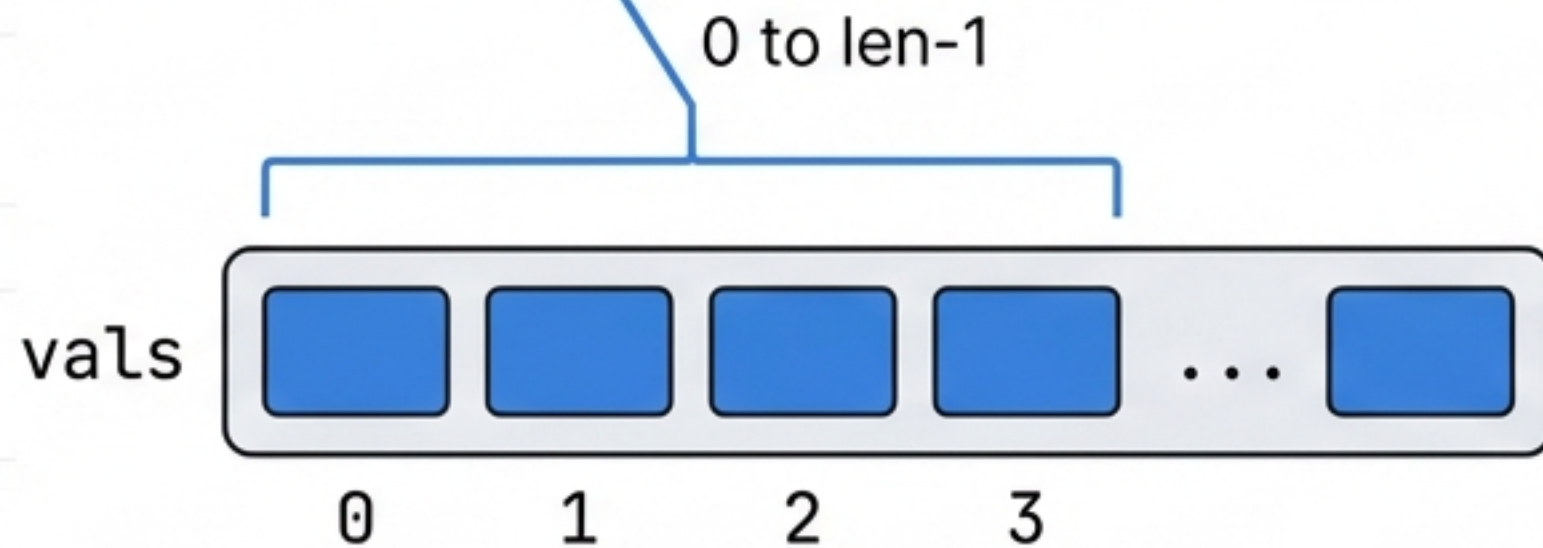
# Creating an integer array with type code 'i'
vals = arr.array('i', [1, 2, 3, 4, 5])
```


Core Operations: Traversal & Iteration

The Range Loop (Index-Based)

Best when you need the index number.

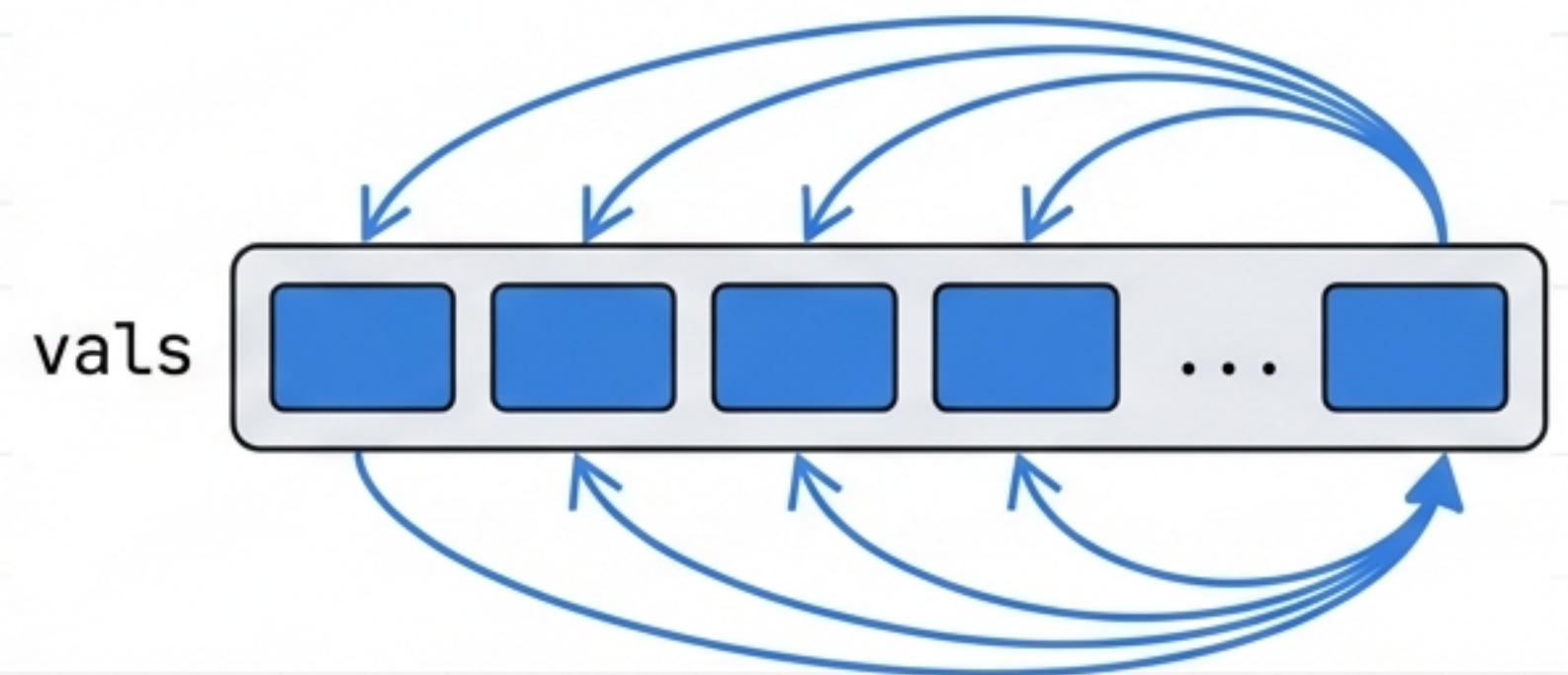
```
for i in range(len(vals)):  
    print(vals[i])
```



The Enhanced Loop (Value-Based)

Cleaner, Pythonic syntax. No index management.

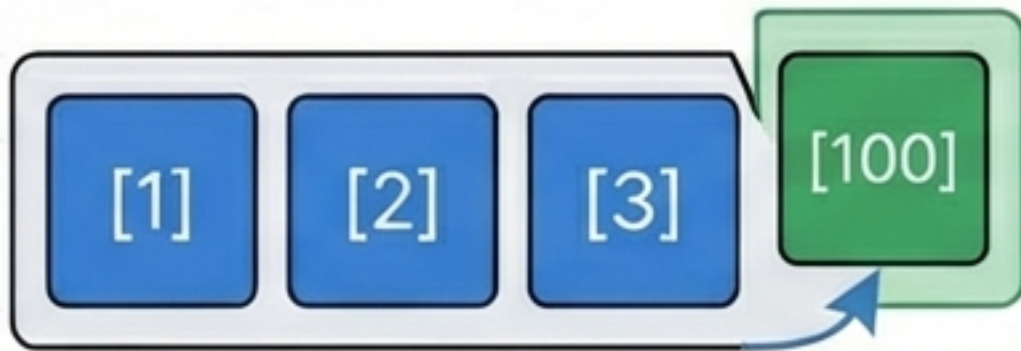
```
for x in vals:  
    print(x)
```



Modification: The Dynamic Nature

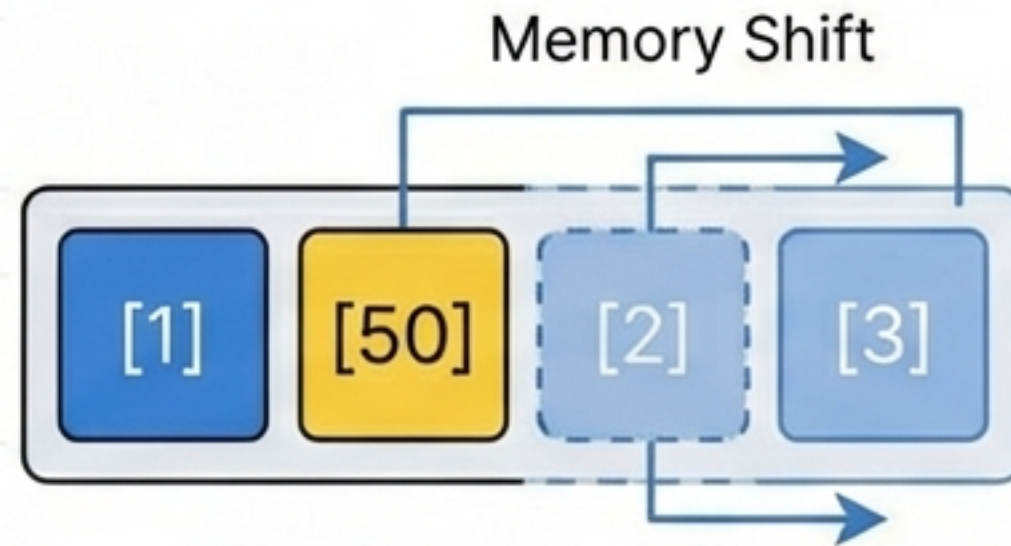
Append

```
vals.append(100)
```



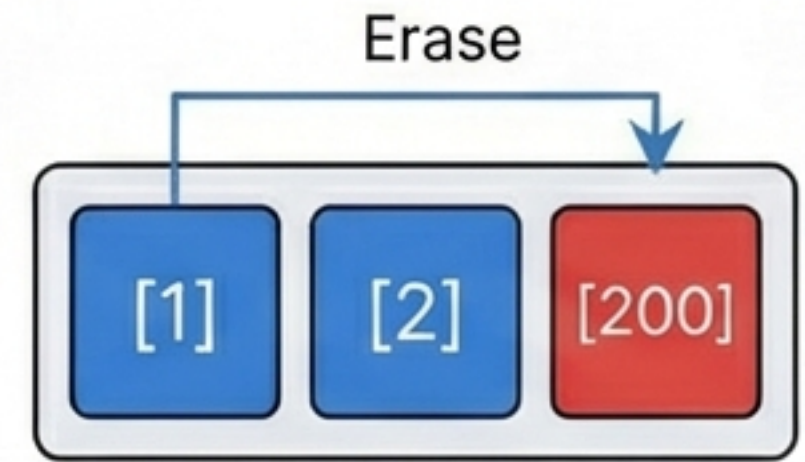
Insert

```
vals.insert(1, 50)
```



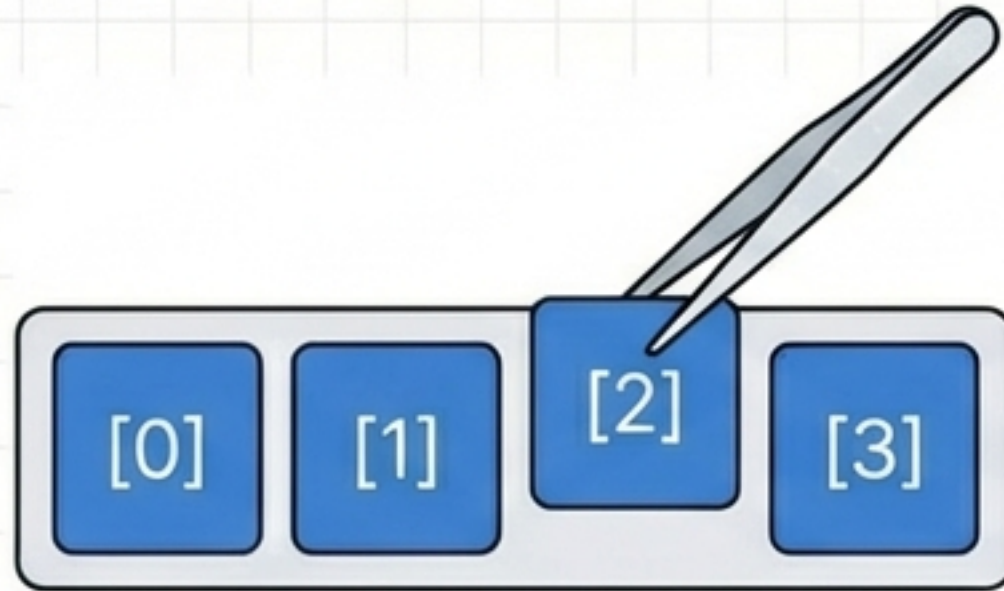
Update

```
vals[2] = 200
```



Surgical Removal vs. LIFO

pop(index)



Removes specific ID.
Returns value.

pop()



No argument = LIFO
(Stack behavior).
Removes last item.

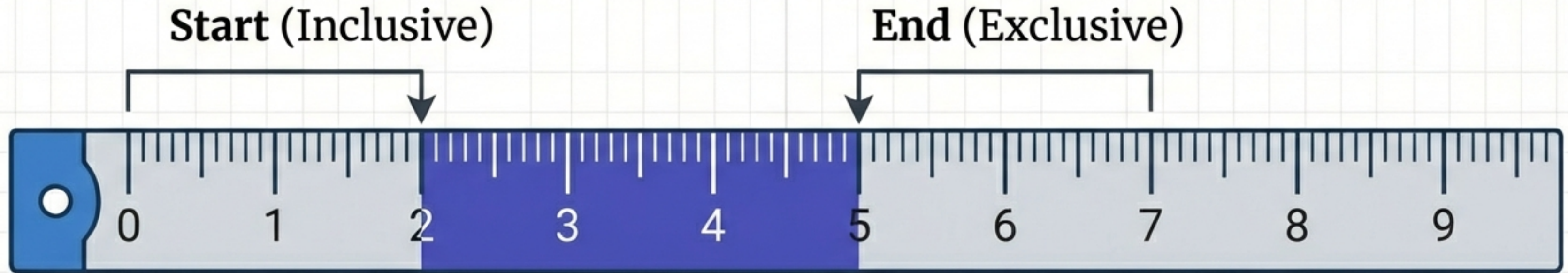
remove(value)



Finds first occurrence
of value. Deletes it.

⚠ Alert: remove() errors if value is missing. pop() errors if array is empty.

The Art of Slicing



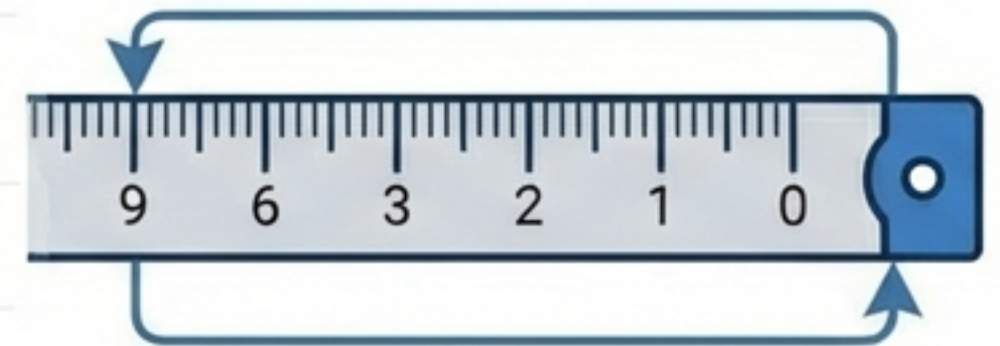
Array[**Start** : **Stop** : Step]

```
vals[2:5]
```

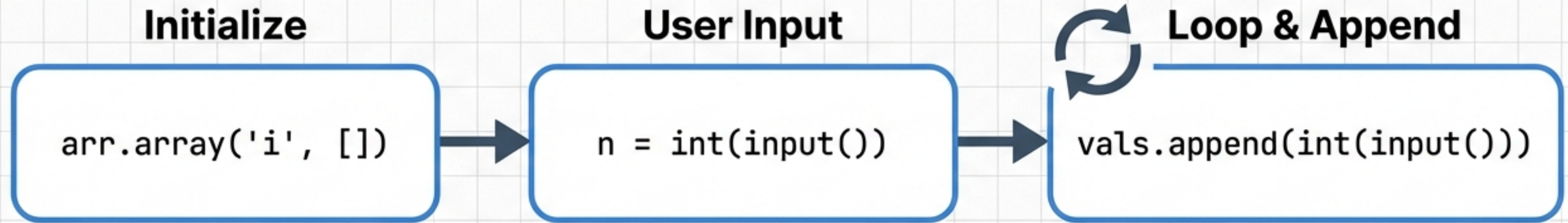
Extracts indices 2, 3, 4

```
vals[::-1]
```

The Magic Trick:
Instant Reversal



Interactivity: Input & Search



Finding Data

```
index = vals.index(45)
```

Returns index of first '45'.
Raises ValueError if not found.



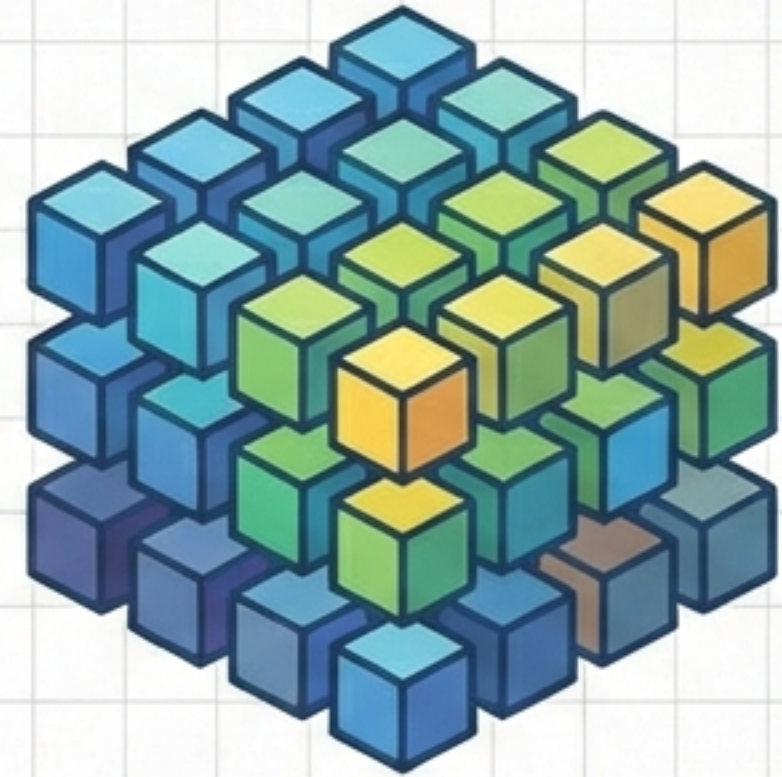
The Upgrade: Enter Numpy



Standard Array Module

1D Only. Strict Memory.

Before and After



Numpy Library

Multi-Dimensional. Heterogeneous.
Scientific Computing.

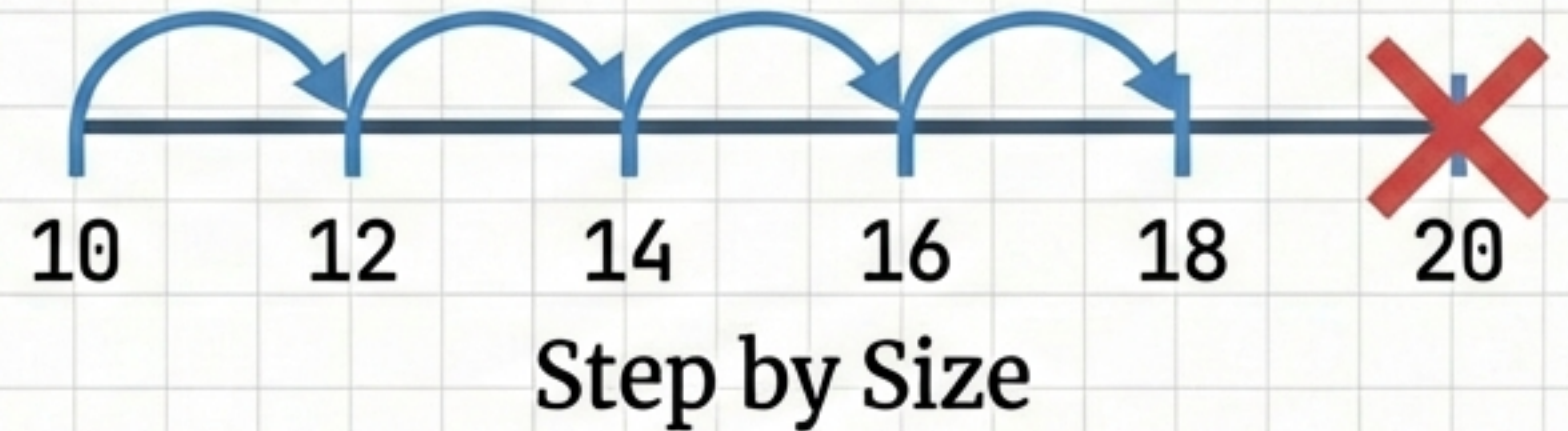
```
pip install numpy  
import numpy as np
```


Numpy Superpowers: Rapid Generation

```
np.linspace(10, 20, 5)
```



```
np.arange(10, 20, 2)
```



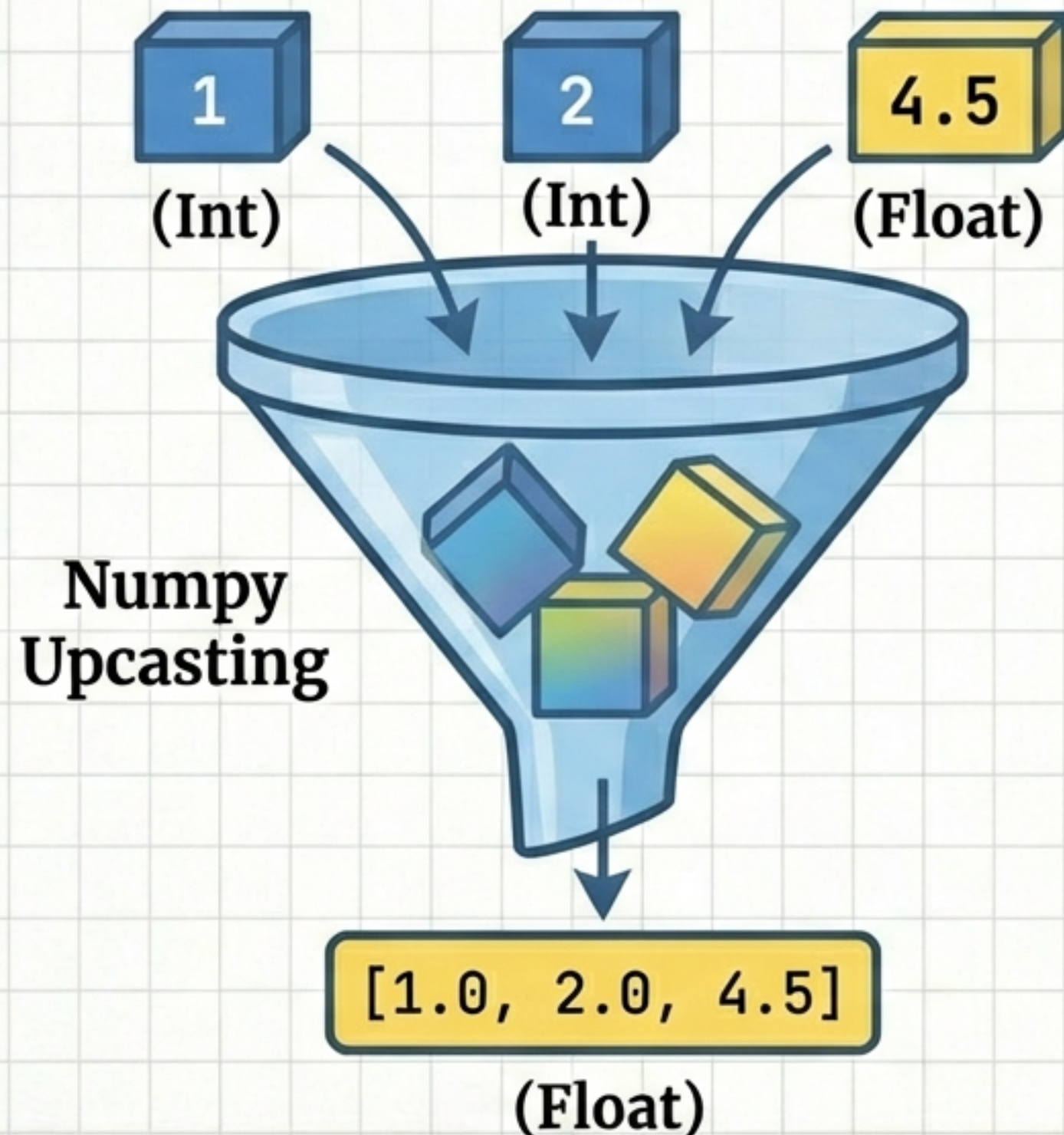
```
np.zeros(5)
```



```
np.full((3), 7)
```



Flexibility: Heterogeneity & Upcasting



💡 **Side Note:** String wins over everything. If 'A' is added, all become Strings. JetBrains Mono

Numpy automatically converts data (Merriweather and **Inter** data to the most complex type to preserve information.

Crossing Dimensions

0D Scalar



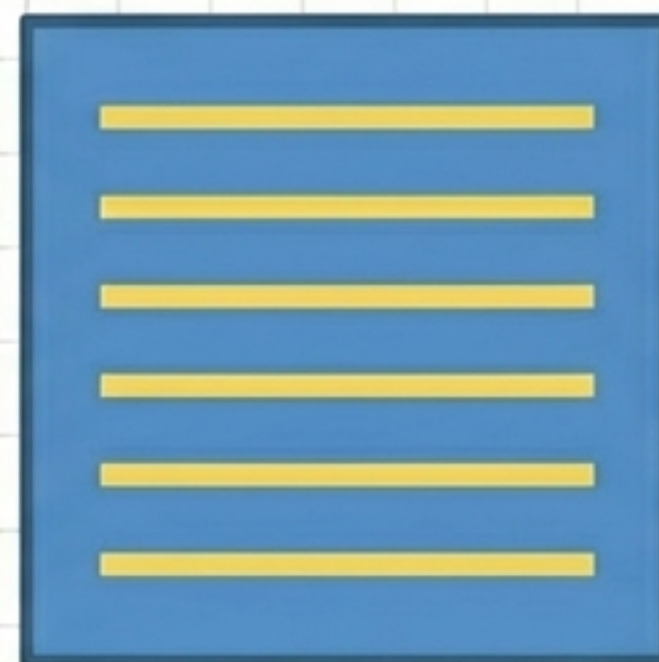
`np.scalar(5)`
JetBrains Mono

1D Vector



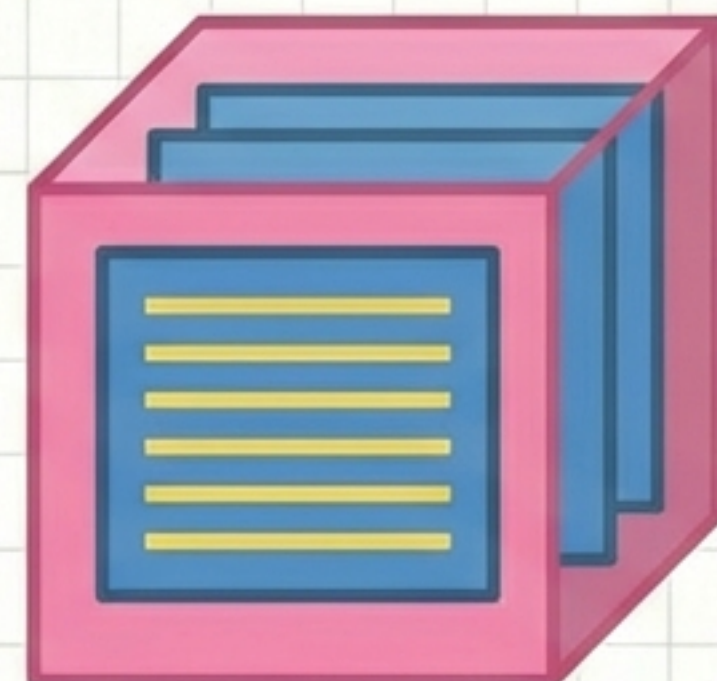
`np.array([1, 2, 3])`
JetBrains Mono

2D Matrix



`np.array([[1, 2],
[3, 4]])`
JetBrains Mono

3D Tensor



`np.array([[[1, 2],
[3, 4]],
[7, 8]])`
JetBrains Mono

Constructing Dimensions in Code

3D Array

```
np.array([  
    # Pink Bracket (Outer)  
    [  
        # Blue Bracket (2D Plane 1)  
        [1, 2], [3, 4] # Yellow Brackets (1D Rows)  
    ],  
    [  
        # Blue Bracket (2D Plane 2)  
        [5, 6], [7, 8] # Yellow Brackets (1D Rows)  
    ]  
])
```

Pink = 3D Container

Blue = 2D Matrix

Yellow = 1D Vector

Visualizing it as 'Arrays inside Arrays' simplifies the syntax.

The Architect's Choice

Standard ``array`` Module

- ✓ Memory Efficiency
- ✓ Simple 1D Structures
- ✓ Strict Type Constraints

Numpy Library

- ✓ Data Science & ML
- ✓ Complex Math & Statistics
- ✓ Matrices (2D, 3D+)
- ✓ Rapid Generation

Practice: Always try to implement raw logic (loops) first to build muscle memory before relying on built-in methods.